

1000.c

```

1  /*****
2  * SUBJECT   : Data Structures & Algorithms (DSA)
3  * TOPIC     : 1D Array Abstract Data Type (ADT)
4  * AUTHOR    : Jotipraksh Zagade
5  * CLASS     : SYBSc.CS (Semester 3)
6  * COLLAGE   : Ahmednagar College
7  * DATE      : 22 July 2026 To 25 July 2026
8  * LANGUAGE  : C
9  *
10 * OPERATIONS IMPLEMENTED:
11 * 1. createArray()           - Allocate heap memory for the array
12 * 2. acceptArray()           - Read initial elements from user
13 * 3. displayArray()          - Output current array elements
14 * 4. append()                - Add element to the end of the array
15 * 5. insert()                - Insert element at index (right shift)
16 * 6. delete()                - Remove element at index (left shift)
17 * 7. get()                   - Return element at specified index
18 * 8. set()                   - Replace element at specified index
19 * 9. Max()                   - Return maximum value in array
20 * 10. Min()                  - Return minimum value in array
21 * 11. sum()                  - Return total sum of elements
22 * 12. recursiveSum()         - Return sum using recursion
23 * 13. average()              - Return average of elements
24 * 14. swap()                 - Helper function to swap two integers
25 * 15. reverse1()             - In-place reversal using two pointers
26 * 16. reverse2()             - Reversal using auxiliary array
27 * 17. leftShift()            - Shift elements left by 1 position
28 * 18. rightShift()           - Shift elements right by 1 position
29 * 19. isSorted()             - Check if array is sorted ascending
30 * 20. insertInSorted()       - Insert into sorted position
31 * 21. rearrange()            - Separate negative numbers to left
32 * 22. appendArray()          - Append source array to target
33 * 23. concatArrays()         - Concatenate two arrays into a third
34 * 24. compareArrays()        - Compare two arrays for equality
35 * 25. copyArray()            - Deep copy of array structure
36 * 26. mergeArrays()          - Merge two sorted arrays
37 * 27. unionSortedArray()     - Set union for sorted arrays
38 * 28. unionUnsortedArray()   - Set union for unsorted arrays
39 * 29. intersectionSortedArray() - Set intersection for sorted arrays
40 * 30. intersectionUnsortedArray() - Set intersection for unsorted arrays
41 * 31. differenceSortedArray() - Set difference for sorted arrays
42 * 32. differenceUnsortedArray() - Set difference for unsorted arrays
43 * 33. setMembershipSortedArray() - Check element presence in sorted set
44 * 34. setMembershipUnsortedArray() - Check element presence in unsorted set
45 *
46 * SEARCH TECHNIQUES:
47 * 35. linearSearch()          - Sequential search, O(n)
48 * 36. transpositionLinearSearch() - Swap found item with predecessor

```

```

49  * 37. moveToFrontLinearSearch() - Move found item to index 0
50  * 38. sentinelLinearSearch() - Search without boundary check in loop
51  * 39. orderedListLinearSearch() - Early stop linear search for sorted lists
52  * 40. binarySearch() - Iterative binary search, O(log n)
53  * 41. rbinSearch() - Recursive binary search, O(log n)
54  *****/
55
56  #include<stdio.h>
57  #include<stdlib.h>
58  #include<stdbool.h>
59
60  typedef struct{
61      int *A;
62      int size;
63      int length;
64  }Array;
65
66  #define IS_INVALID(arr)((arr) == NULL || (arr)->A == NULL)
67
68  void createArray(Array *arr){
69      if(arr == NULL){
70          printf("Error : Array pointer is NULL.\n");
71          exit(EXIT_FAILURE);
72      }
73      printf("Enter size of the array : ");
74      if(scanf("%d",&arr->size) != 1 || arr->size <= 0){
75          printf("Invalid Capacity/size entered.\n");
76          exit(EXIT_FAILURE);
77      }
78
79      arr->A = (int*)malloc(arr->size*sizeof(int));
80      if(arr->A == NULL){
81          printf("Memory allocation failed.\n");
82          exit(EXIT_FAILURE);
83      }
84      arr->length = 0;
85  }
86
87  void acceptArray(Array *arr){
88      if(IS_INVALID(arr) || arr->size <= 0) exit(EXIT_FAILURE);
89      int n;
90      printf("Enter total elements in the array : ");
91      if(scanf("%d",&n) != 1)
92          exit(EXIT_FAILURE);
93
94      if(n <= 0){
95          printf("No elements added.\n");
96          return;
97      }
98      if(n > arr->size){

```

```
99     printf("Error : Number of elements(%d) exceeds max size(%d).\n",n,arr->size);
100     printf("Truncating input size to max size(%d).\n",arr->size);
101     n = arr->size;
102 }
103 printf("Enter %d elements of the array : ",n);
104 for(int i = 0; i < n; i++){
105     if(scanf("%d", &arr->A[i]) == 1)
106         arr->length++;
107     else{
108         printf("Invalid integer input encountered at index %d. Stopping reading.\n",i);
109         break;
110     }
111 }
112 }
113
114 void displayArray(Array *arr){
115     if(IS_INVALID(arr) || arr->size <= 0) exit(EXIT_FAILURE);
116
117     if(arr->length > 0){
118         for(int i = 0; i < arr->length; i++){
119             printf("%d ",arr->A[i]);
120         }
121         printf("\n");
122     }
123 }
124
125 void append(Array *arr, int x){
126     if(IS_INVALID(arr))return;
127
128     if(arr->length < arr->size){
129         arr->A[arr->length++] = x;
130     }
131 }
132
133 void insert(Array *arr, int index, int x){
134     if(IS_INVALID(arr))return;
135
136     if(index >= 0 && index <= arr->length && arr->length < arr->size){
137         for(int i = arr->length; i > index; i--){
138             arr->A[i] = arr->A[i - 1];
139         }
140         arr->A[index] = x;
141         arr->length++;
142     }
143 }
144
145 int delete(Array *arr, int index){
146     if (IS_INVALID(arr) || index < 0 || index >= arr->length) {
147         printf("Invalid index for deletion.\n");
148         return -1;
```

```
149     }
150     int deleted = arr->A[index];
151     for(int i = index; i < arr->length - 1; i++){
152         arr->A[i] = arr->A[i + 1];
153     }
154     arr->length--;
155     return deleted;
156 }
157 /*
158  * NOTE ON DELETION & LOW-LEVEL MEMORY:
159  *
160  * 1. Physical Memory Reality:
161  *   At the hardware level, memory is never truly "deleted" or "cleared".
162  *   Deletion is merely a logical operation executed by left-shifting subsequent
163  *   elements and decrementing 'length'. The value at index = length remains in RAM,
164  *   but 'length' acts as a boundary gatekeeper—treating everything past length - 1
165  *   as invalid/garbage memory.
166  *
167  * 2. Abstraction & Encapsulation:
168  *   High-level languages (C++ STL vector, Java ArrayList, Python list) perform
169  *   this exact low-level copy and shift routine under the hood. They encapsulate
170  *   the raw buffer and length counter inside a class, exposing clean methods
171  *   like delete() or pop() to hide raw memory management from the user.
172  *
173  * 3. Primitive Hardware Operations:
174  *   Fundamentally, CPU/Hardware only understands memory addressing, data copying,
175  *   and arithmetic/logic operations. All abstract operations (insert, append, delete)
176  *   are built on top of these primitives.
177  */
178 int get(Array *arr, int index){
179     if (IS_INVALID(arr) || index < 0 || index >= arr->length) {
180         return -1;
181     }
182     return arr->A[index];
183 }
184
185 void set(Array *arr, int index, int x){
186     if (IS_INVALID(arr) || index < 0 || index >= arr->length) {
187         printf("Invalid index for set operation.\n");
188         return;
189     }
190     arr->A[index] = x;
191 }
192
193 int Max(Array *arr) {
194     if (IS_INVALID(arr) || arr->length == 0) return -1;
195
196     int max = arr->A[0];
197     for (int i = 1; i < arr->length; i++) {
198         if (arr->A[i] > max)
```

```
199         max = arr->A[i];
200     }
201     return max;
202 }
203
204 int Min(Array *arr) {
205     if (IS_INVALID(arr) || arr->length == 0) return -1;
206
207     int min = arr->A[0];
208     for (int i = 1; i < arr->length; i++) {
209         if (arr->A[i] < min)
210             min = arr->A[i];
211     }
212     return min;
213 }
214
215 int sum(Array *arr){
216     if (IS_INVALID(arr) || arr->length == 0) return 0;
217
218     int total = 0;
219     for (int i = 0; i < arr->length; i++) {
220         total += arr->A[i];
221     }
222     return total;
223 }
224
225 int recursiveSum(Array *arr, int i){
226     if(i >= arr->length)
227         return 0;
228     return arr->A[i] + recursiveSum(arr, i + 1);
229 }
230
231 float avarage(Array *arr){
232     if (IS_INVALID(arr) || arr->length <= 0) return 0.0f;
233     return (float)sum(arr) / arr->length;
234 }
235
236 void swap(int *x, int *y){
237     int temp = *x;
238     *x = *y;
239     *y = temp;
240 }
241
242 void reverse1(Array *arr){
243     if (IS_INVALID(arr) || arr->length <= 0) return;
244
245     for (int i = 0, j = arr->length - 1; i < j; i++, j--) {
246         swap(&arr->A[i], &arr->A[j]);
247     }
248 }
```

```
249
250 void reverse2(Array *arr){
251     if (IS_INVALID(arr) || arr->length <= 0) return;
252
253     int *B = (int*)malloc(arr->size * sizeof(int));
254     if (B == NULL) {
255         printf("Memory allocation failed during reversing the array.\n");
256         return;
257     }
258     for (int i = 0, j = arr->length - 1; i < arr->length; i++, j--) {
259         B[i] = arr->A[j];
260     }
261     for (int i = 0; i < arr->length; i++) {
262         arr->A[i] = B[i];
263     }
264     free(B);
265 }
266
267 void leftShift();
268
269 void rightshift();
270
271 bool isSorted(Array *arr){
272     if (IS_INVALID(arr) || arr->length <= 1) return true;
273 }
274
275
276 void insertInSorted(Array *arr, int x);
277
278 void rearrange(Array *arr);
279
280 void appendArray(Array *target, Array *source);
281
282 void concatArrays(Array *arr1, Array arr2, Array arr3);
283
284 void compareArrays(Array *arr1, Array arr2);
285
286 void copyArray(Array *arr);
287
288 void mergeArrays(Array *arr1, Array *arr2);
289
290 void unionSortedArray();
291
292 void unionUnsortedArray();
293
294 void intersectionSortedArray();
295
296 void intersectionUnsortedArray();
297
298 void differenceSortedArray();
```

```
299
300 void differenceUnsortedArray();
301
302 void setMembershipSortedArray();
303
304 void setMembershipUnsortedArray();
305
306 int linearSearch(Array *arr, int key){
307     if (IS_INVALID(arr) || arr->length <= 0) return -1;
308     for(int i = 0; i < arr->length; i++){
309         if(arr->A[i] == key)
310             return i;
311     }
312     return -1;
313 }
314
315 //variation of probabiltiy search
316 int transpositionLinearSearch(Array *arr, int key) {
317     if (IS_INVALID(arr) || arr->length <= 0) return -1;
318     for (int i = 0; i < arr->length; i++) {
319         if (arr->A[i] == key) {
320             if (i > 0) {
321                 swap(&arr->A[i], &arr->A[i - 1]);
322                 return i - 1;
323             }
324             return i;
325         }
326     }
327     return -1;
328 }
329
330 //probability search variation
331 int moveToFrontLinearSearch(Array *arr, int key) {
332     if (IS_INVALID(arr) || arr->length <= 0) return -1;
333     for (int i = 0; i < arr->length; i++) {
334         if (arr->A[i] == key) {
335             if (i > 0) {
336                 swap(&arr->A[i], &arr->A[0]);
337                 return 0;
338             }
339             return i;
340         }
341     }
342     return -1;
343 }
344
345 int sentinelLinearSearch(Array *arr, int key) {
346     if (IS_INVALID(arr) || arr->length <= 0) return -1;
347
348     int lastElement = arr->A[arr->length - 1];
```

```
349     arr->A[arr->length - 1] = key;
350
351     int i = 0;
352     while (arr->A[i] != key) {
353         i++;
354     }
355     arr->A[arr->length - 1] = lastElement;
356
357     if (i < arr->length - 1 || key == lastElement)
358         return i;
359
360     return -1;
361 }
362
363 int orderedListLinearSearch(Array *arr, int key) {
364     if (IS_INVALID(arr) || arr->length <= 0) return -1;
365     for (int i = 0; i < arr->length; i++) {
366         if (key == arr->A[i])
367             return i;
368         if (arr->A[i] > key)
369             return -1;
370     }
371     return -1;
372 }
373
374 int binarySearch(Array *arr, int key) {
375     if (IS_INVALID(arr) || arr->length <= 0) return -1;
376
377     int low = 0;
378     int high = arr->length - 1;
379     while (low <= high) {
380         int mid = low + (high - low) / 2;
381         if (arr->A[mid] == key)
382             return mid;
383         else if (arr->A[mid] < key)
384             low = mid + 1;
385         else
386             high = mid - 1;
387     }
388     return -1;
389 }
390
391 int rbinSearch(Array *arr, int low, int high, int key) {
392     if (IS_INVALID(arr) || low > high)
393         return -1;
394
395     int mid = low + (high - low) / 2;
396
397     if (arr->A[mid] == key)
398         return mid;
```



```
399     else if (arr->A[mid] < key)
400         return rbinSearch(arr, mid + 1, high, key);
401     else
402         return rbinSearch(arr, low, mid - 1, key);
403 }
404
405 int main(){
406
407     return 0;
408 }
```